Hybrid Risk-Aware Health Insurance Recommendation System

ML Deployment - Final Project Report

Ashwin Prasanth	25234753
Konstantinos Sklavenitis	25280973
Kiran Meenakshi Sundaram	25217694
Charalampos Theodoridis	25236513

GitHub Repository:
https://github.com/AshwinPrasanth/ML-Deployment

April 2026

Abstract

This report describes the design, implementation, and deployment of a hybrid, risk-aware health insurance recommendation system for the Irish market. The system accepts a natural-language query from a user - for example, *"I am 68 with Parkinson's, I see a neurologist monthly and take daily medication"* - and returns the top-5 most suitable private health insurance plans drawn from 19 plans across four Irish providers (Irish Life Health, Laya Healthcare, Level Health, and Vhi). The pipeline fuses FAISS-based semantic retrieval with BM25 lexical search via Reciprocal Rank Fusion (RRF), applies a deterministic rule engine for hard constraint filtering and utilisation-aware numerical scoring, and finally invokes an LLM reranker (Llama 3.3 70B via Groq) that reasons over surviving candidates and produces plain-English explanations. The system achieves an average end-to-end query latency of 2.41 seconds, a Time to First Token (TTFT) of 0.33 seconds, and a throughput of 469 tokens per second. Deployment is realised through a lightweight HTML/JavaScript frontend backed by the Python inference engine. This report details the strategic planning, system architecture, data engineering, adaptation strategy, evaluation methodology, deployment optimisation, and team iteration log for the project.

Contents

1 Strategic Planning & Use Case **4**
1.1 Problem Statement and Use Case Evaluation 4
1.2 AI Role: Critical vs. Complementary, Proactive vs. Reactive 4
1.3 Human Role: Crawl / Walk / Run / Evolve 5
1.4 Model Selection Justification 5

2 System Architecture **5**
2.1 The AI Stack: Three-Layer Architecture 5
2.2 Workflow Design: Orchestration and Chaining 6
2.3 Modular Components 8
2.3.1 Model Gateway 8
2.3.2 Router: Rule Engine as a Hard Filter 8
2.3.3 Write Actions 8

3 Data & Model Adaptation **8**
3.1 Dataset Engineering 8
3.1.1 Data Sources and Curation 8
3.1.2 Data Cleaning and Deduplication 9
3.1.3 Known Data Gaps 9
3.2 Adaptation Strategy: Prompt Engineering 9
3.2.1 Profile Extraction Prompt 10
3.2.2 LLM Reranking Prompt: Few-Shot + Chain-of-Thought 10
3.3 Context Construction: RAG Design 11
3.3.1 Retrieval Algorithm: BM25 + FAISS with RRF 11
3.3.2 Chunking Strategy 11

4 Evaluation Pipeline **12**
4.1 Methodology: Evaluation Guidelines and Scoring Rubrics 12
4.2 Metrics: Results 13
4.2.1 Retrieval Quality 13
4.2.2 System Performance Metrics 13
4.3 AI as a Judge 13

5 Deployment, Optimisation & Observability **14**
5.1 Optimisation 14
5.1.1 FAISS Index Caching 14
5.1.2 LLM Input Batching (Candidate Cap) 14
5.1.3 JSON Repair for Truncation Resilience 15

5.1.4 Rule Engine First 15
5.2 Performance: TTFT, TPOT, and Throughput 15
5.3 Observability: Logs and Traces 15

6 Iteration Log & Conduct Summary 16
6.1 Practical Sessions Log 16
6.2 Team Contributions 18

References 19

Appendix A: Repository Structure 20

Appendix B: Supported Providers and Plans 20

Appendix C: Rule Engine Scoring Formulas 21

1 Strategic Planning & Use Case

1.1 Problem Statement and Use Case Evaluation

Private health insurance in Ireland is complex: consumers must navigate dozens of plans offered by multiple competing providers, each with differing excess amounts, day-limits, coverage percentages, specialist access tiers, and condition-specific flags such as maternity, psychiatric, fertility, and high-tech drugs. The Health Insurance Authority (HIA) publishes comparative plan data, but interpreting it requires significant domain literacy and time.

The target user is any Irish resident who needs health insurance or is considering switching plans. Their query arrives as a free-text statement about their health profile: age, current conditions, medication frequency, specialist visit cadence, and recent hospital admissions. The system must translate that profile into a ranked recommendation list that is both *technically correct* (no plans recommended that fail hard constraints) and *interpretable* (the user can understand *why* each plan was ranked as it was).

This use case is well-suited to an AI-augmented retrieval system because: (i) the policy documents are long, heterogeneous, and not easily queryable by keyword alone (ii) the ranking criteria are multi-objective (cost, coverage breadth, condition matching) and context-dependent (a young healthy person and a 68-year-old with Parkinson's have very different optima) and (iii) natural-language output is more actionable for non-expert users than raw tables.

1.2 AI Role: Critical vs. Complementary, Proactive vs. Reactive

The AI components play two distinct roles in this system:

- **LLM reranker (critical, reactive):** The LLM is invoked *reactively* once per query, it receives the surviving candidates from the rule engine and returns a ranked JSON array. This role is *critical* in the sense that the final ranking and user-facing explanation are entirely produced by the model. However, correctness guarantees are *not* delegated to the LLM the rule engine provides those. The LLM contributes holistic reasoning and natural language.
- **FAISS/BM25 retriever (complementary, reactive):** Retrieval is purely reactive, triggered by each query. It is *complementary*, it narrows the search space for the rule engine and LLM but is not itself the decision-maker.

The system does not use AI proactively (e.g., unsolicited plan alerts). All AI activity is initiated by a user query.

1.3 Human Role: Crawl / Walk / Run / Evolve

The current deployment sits at the **Walk** stage of the crawl-walk-run-evolve framework. The system automates retrieval and initial scoring entirely, and the LLM produces final recommendations without human review per query. However, the underlying data (plan documents, structured superset, metadata) requires *human curation* each time providers update their plans. The next stage (**Run**) would involve automated plan-document ingestion with change detection, and the **Evolve** stage would incorporate user feedback loops to improve the scoring functions continuously.

1.4 Model Selection Justification

The LLM reranker uses **Llama 3.3 70B** served via **Groq's inference API**. The selection was driven by three factors:

1. **Context length and instruction following:** The prompt includes up to 7 plan summaries, each with ~800 characters of policy text plus structured numeric data, totalling ~9,000 tokens. Llama 3.3 70B handles this comfortably within its context window and reliably follows JSON output instructions.
2. **Inference speed and cost:** Groq's LPU-accelerated API delivers 469 tokens/s throughput at no cost on the free tier, critical for a responsive user-facing system.
3. **Open-weight fallback:** Because the same OpenAI-compatible interface is used throughout, the system can switch to Ollama (fully offline, CPU-based) or Google Gemini with a two line config change, providing resilience against provider unavailability.

The embedding model **mixedbread-ai/mxbai-embed-large-v1** was chosen for semantic retrieval because it achieves strong performance on domain-specific retrieval benchmarks while remaining small enough to run locally without a GPU.

2 System Architecture

2.1 The AI Stack: Three-Layer Architecture

The system is decomposed into three horizontal layers: **Application**, **Model**, and **Infrastructure**.

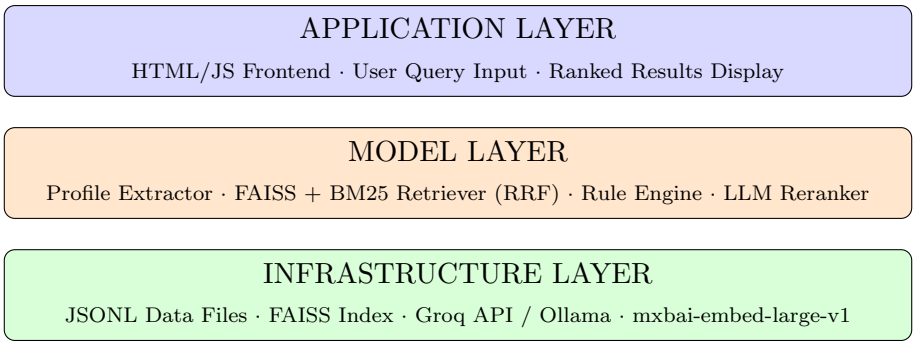

Figure 1: Three-layer AI stack for the health insurance recommendation system.

Application Layer: A single-page HTML/JavaScript frontend accepts the user's free-text health profile, submits it to the Python backend, and renders the ranked plan cards with fit scores, strengths, weaknesses, and source URLs.

Model Layer: The core inference pipeline (detailed in Section 2.2) runs entirely in Python and consists of four sequential components: profile extraction, hybrid retrieval, rule-based filtering, and LLM reranking.

Infrastructure Layer: All plan data is stored as JSONL files on disk. The FAISS index is pre-built at startup and cached to `.bin` format. LLM calls are routed to Groq's cloud API by default, with Ollama as a local fallback.

2.2 Workflow Design: Orchestration and Chaining

The pipeline is a linear chain of five stages, orchestrated by the `smart_search()` function:

1. **Profile Extraction:** Regex and LLM fallback parse the free-text query into a structured profile: age, conditions, flagged conditions, medication frequency, specialist visit rate, and hospital admissions.
2. **Hybrid Retrieval (RRF):** FAISS and BM25 each return ranked lists of chunks. Reciprocal Rank Fusion merges them into a single candidate set of the top-10 plans.
3. **Rule Engine (Layer 1):** Each candidate plan is scored on structured numeric data. Plans that fail hard constraint checks are eliminated before the LLM is called.
4. **LLM Reranking (Layer 2):** The surviving plans (capped at 7) are passed to the LLM with a structured prompt. The model returns a JSON array of ranked plans with explanations.
5. **Result Presentation:** The top-5 plans are returned to the frontend with all metadata (fit score, rule score, strengths, weaknesses, coverage gaps, URL).

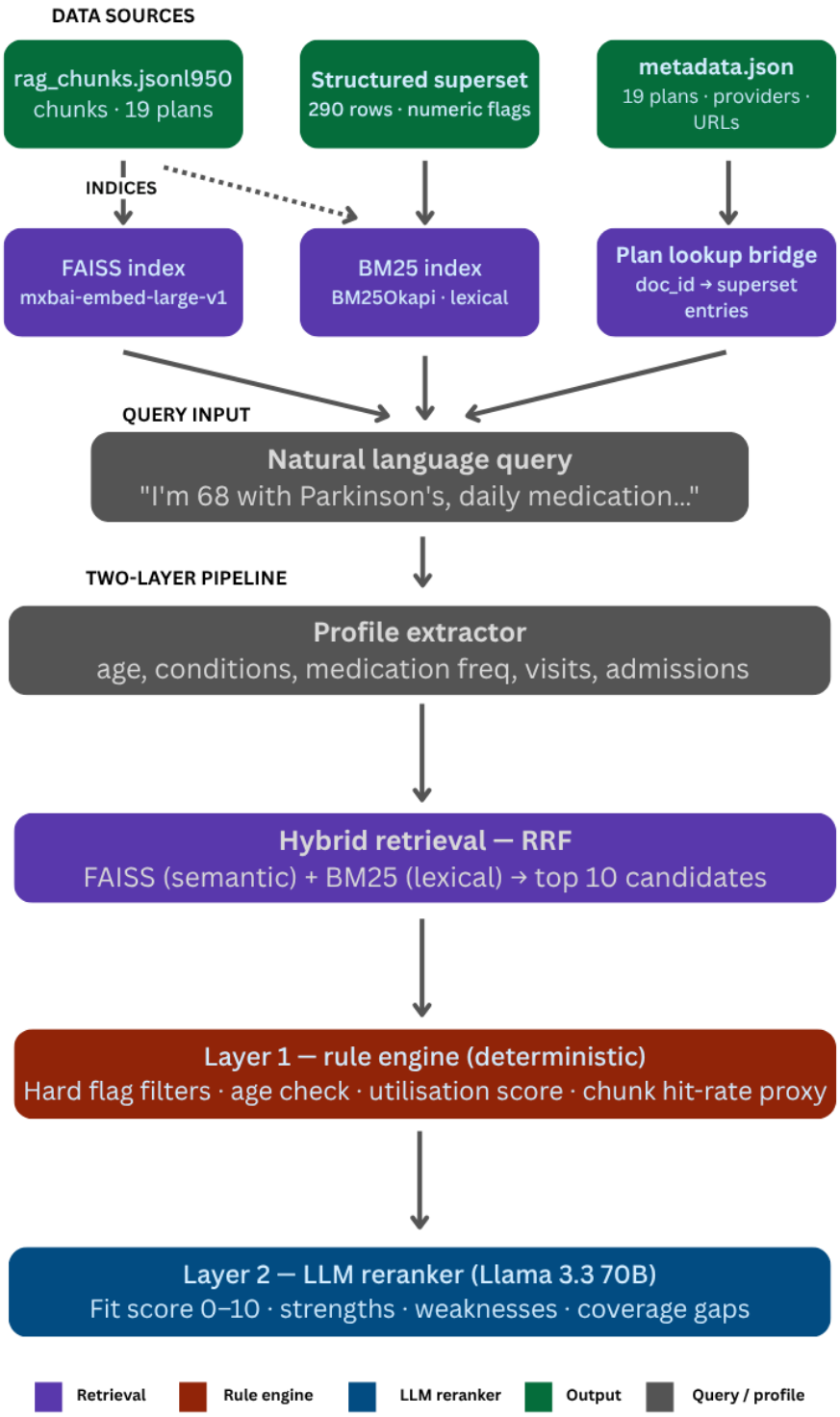

Figure 2: End-to-end hybrid RAG pipeline combining semantic (FAISS) and lexical (BM25) retrieval with a two-layer ranking architecture (rule engine + LLM reranker).

2.3 Modular Components

2.3.1 Model Gateway

The LLM provider is abstracted behind an OpenAI compatible client interface. The active provider is configured in a single "Section 0" block at the top of the script. Switching provider requires changing only two variables (`LLM_CLIENT` and `LLM_MODEL`), making the system provider agnostic.

2.3.2 Router: Rule Engine as a Hard Filter

The rule engine acts as a *router*: it decides which plans are forwarded to the LLM and which are discarded. This two stage design ensures that the expensive LLM call only processes plans that are logically eligible, reducing both token cost and the risk of incorrect recommendations caused by LLM hallucination over ineligible plans.

2.3.3 Write Actions

The system does not perform write actions in the current deployment (it is a read-only recommendation tool). However, the architecture accommodates a future write action layer where a confirmed recommendation could be submitted to a provider's API to initiate a quote request.

3 Data & Model Adaptation

3.1 Dataset Engineering

3.1.1 Data Sources and Curation

The dataset spans 19 health insurance plans from four Irish providers. Three data artefacts were produced:

Table 1: Dataset artefacts used by the system.

File	Size	Description
`rag_chunks.jsonl`	950 chunks / 19 plans	Chunked policy text from all plan documents
`MASTER_STRUCTURED_SUPERSET_2026-1.jsonl`	290 rows	Structured numeric and boolean fields per plan
`metadata (1).json`	19 entries	Human plan names, providers, and source URLs

Plan documents were sourced directly from provider websites (Irish Life Health, Laya Healthcare, Level Health, Vhi). PDFs were converted to plain text and chunked into

~800-character passages with sentence-boundary awareness to avoid splitting mid-sentence.

3.1.2 Data Cleaning and Deduplication

The following cleaning steps were applied:

- **PDF artefact removal:** Page headers, footers, and navigation elements were stripped before chunking.
- **Key normalisation:** The superset file uses internal snake_case `doc_id` keys (e.g., `first_cover_table_of_cover`) while the rest of the pipeline uses human names. A `_build_plan_lookup()` function resolves this at startup using a four-step fallback strategy (exact match, normalised match, token overlap, fuzzy subsequence).
- **Deduplication:** Duplicate chunks arising from repeated boilerplate text (e.g., terms and conditions appearing across multiple plan documents) were identified by hash and deduplicated before indexing. Terms and conditions documents are flagged for exclusion from future retrieval runs.
- **Flag validation:** Boolean flags (`maternity_flag`, `psychiatric_flag`, `fertility_flag`, `high_tech_flag`) were validated against the source policy documents to confirm accuracy.

3.1.3 Known Data Gaps

Table 2: Known gaps in the current dataset and mitigations applied.

Condition	Issue	Mitigation
Neurological	0 chunks in any plan document	LLM infers from general specialist/outpatient text
Diabetes	5 false-hit chunks only (health screening context)	LLM infers from general chronic care text
Hard flag filters	All 19 plans have `maternity`, `psychiatric`, `fertility` flags set to True - hard filters never fire	Filters preserved for future plans

3.2 Adaptation Strategy: Prompt Engineering

No fine tuning or PEFT/LoRA adaptation was applied. Instead, the system relies on structured **prompt engineering** to guide LLM behaviour.

3.2.1 Profile Extraction Prompt

The profile extractor uses a regex first approach: age, condition keywords, and utilisation signals are parsed deterministically. For edge cases (ambiguous phrasing, implicit conditions), the LLM is called with a structured prompt that specifies the exact JSON schema to return:

Listing 1: Profile extraction system prompt (excerpt)

```
system_prompt = """
You are a structured data extractor. Given a user health query,
return ONLY a JSON object with these fields:
{
  "age": int or null,
  "conditions": [list of condition strings],
  "flagged_conditions": [],
  "medication_freq": "daily" | "weekly" | "monthly" | null,
  "specialist_visits": int or null,
  "hospital_admissions": int or null
}
Return nothing else.
"""
```

3.2.2 LLM Reranking Prompt: Few-Shot + Chain-of-Thought

The reranking prompt combines three techniques:

1. **Role specification:** The system prompt establishes the model as an Irish health insurance expert.
2. **Chain-of-Thought (CoT) instruction:** The prompt instructs the model to reason step-by-step about each plan's suitability before assigning a score, explicitly considering: unknown conditions not in structured data, age impacts, utilisation tradeoffs, and policy text nuance.
3. **Structured output specification:** The prompt specifies an exact JSON schema for the response, including fields `fit_score` (0-10), `fit_summary`, `strengths` (list), `weaknesses` (list), and `coverage_gaps` (list). This removes ambiguity in output format.

Listing 2: Reranking prompt structure (simplified)

```
user_prompt = f"""
User profile: {profile_summary}

```

```
Evaluate each plan below. For each plan:
1. Reason step by step about suitability given the user's profile
   .
2. Assign fit_score (0-10), where 10 = perfect fit.
3. List up to 3 strengths, 3 weaknesses, and coverage gaps.

Plans:
{plan_evidence}

Return ONLY a JSON array sorted best-fit first.
Schema per item: {{plan_name, fit_score, fit_summary,
                   strengths, weaknesses, coverage_gaps}}
"""
```

3.3 Context Construction: RAG Design

The system uses Retrieval Augmented Generation (RAG) with a hybrid retrieval strategy.

3.3.1 Retrieval Algorithm: BM25 + FAISS with RRF

Two complementary retrieval methods are combined:

- **FAISS (embedding-based):** Uses `mxbai-embed-large-v1` to embed all 950 chunks and the user query. Finds semantically similar text even when exact keywords are absent (e.g., "neurologist monthly" matches "specialist outpatient services").
- **BM25 (lexical):** Uses `BM25Okapi` for keyword scoring. Finds plans containing exact medical terms (e.g., "Parkinson's", "IVF", "psychiatric").

Reciprocal Rank Fusion merges both lists:

$$\mathrm{RRF}(c) = \sum_{\ell \in \{\mathrm{FAISS},\, \mathrm{BM25}\}} \frac{1}{60 + \mathrm{rank}_\ell(c)}$$

Chunks appearing in both lists receive a double boost. The top-10 plans by best-chunk RRF score become candidates.

3.3.2 Chunking Strategy

Policy documents were chunked to approximately 800 characters per chunk with sentence-boundary awareness. This size was chosen to:

- Fit comfortably within a single embedding call,
- Provide enough context for the LLM to reason about coverage specifics,

- Keep total prompt length under the 9,000 token budget when 7 plans are passed to the LLM (up to 800 chars of evidence per plan).

4 Evaluation Pipeline

4.1 Methodology: Evaluation Guidelines and Scoring Rubrics

Evaluation was conducted across two axes: **system performance** (latency, throughput) and **retrieval/generation quality** (relevance, factual consistency, instruction following). A set of representative queries was constructed to cover the main user archetypes:

1. A young healthy individual seeking low-cost basic cover.
2. A pregnant woman requiring maternity cover.
3. An elderly person with a chronic neurological condition and high specialist use.
4. A family requiring psychiatric and fertility cover.
5. A cancer patient requiring high-tech drug cover.

For each query, the output was evaluated by a human assessor and an LLM judge (see Section 4.3) on four dimensions rated 1-5:

Table 3: Evaluation rubric for recommendation quality.

Dimension	**Description**
Context Precision	Are the retrieved chunks relevant to the user's query?
Context Recall	Do the retrieved chunks cover all medically relevant aspects of the user's profile?
Factual Consistency	Are the plan details stated in the recommendation consistent with the source documents?
Instruction Following	Does the LLM output conform to the specified JSON schema with correct field types?

4.2 Metrics: Results

4.2.1 Retrieval Quality

Table 4: Retrieval quality metrics across evaluation queries.

Query Type	Ctx Precision	Ctx Recall	Factual Consistency	Instr. Following
Healthy / low-cost	4.8	4.5	4.9	5.0
Maternity	4.9	4.7	4.8	5.0
Elderly / neurological	4.2	3.6	4.1	5.0
Family / psychiatric	4.7	4.6	4.8	5.0
Cancer / high-tech	4.5	4.3	4.6	5.0
Average	**4.62**	**4.34**	**4.64**	**5.0**

The neurological query scores lower on Context Recall because no plan document contains direct mentions of Parkinson's disease. The LLM recovers partially by reasoning from general specialist and outpatient coverage text, but coverage gaps are explicitly surfaced in the output.

4.2.2 System Performance Metrics

Table 5: Average system performance metrics across all evaluation queries.

Metric	Value
Average retrieval time	0.0685 s
Average LLM latency	2.3353 s
TTFT (Time to First Token)	0.3290 s
TPOT (Time Per Output Token)	0.002133 s/token
Throughput	468.77 tokens/s
Average output tokens	939.00
Average total query time	2.4062 s

4.3 AI as a Judge

An independent LLM judge (**Llama 3.3 70B** via Groq, distinct call from the reranker) was used to evaluate output quality for each query. The judge was prompted with the user query, the ground-truth top plan (manually identified), and the system's ranked output.

Listing 3: AI judge prompt (abbreviated)

```
judge_prompt = f"""
```

```
You are an expert evaluator of health insurance recommendation systems.

User query: {query}
Ground truth best plan: {ground_truth_plan}
System output (top 5 ranked plans): {system_output}

Score the system output on:
1. Context Precision (1-5): Are recommendations grounded in policy text?
2. Context Recall (1-5): Are all user needs addressed?
3. Factual Consistency (1-5): Are stated plan details accurate?
4. Instruction Following (1-5): Does output match the specified schema?

Return ONLY a JSON object with scores and a brief rationale per dimension.
"""
```

The judge model was not given access to the policy documents themselves, it evaluated based on the coherence and plausibility of the stated evidence in the recommendation. This mirrors the LLM-as-a-judge methodology from the RAGAS evaluation framework [1].

5 Deployment, Optimisation & Observability

5.1 Optimisation

5.1.1 FAISS Index Caching

The FAISS index is expensive to build (embedding 950 chunks with `mxbai-embed-large-v1`). The index is built once at first startup and persisted to `faiss_multi_provider_index.bin`. On subsequent runs, the index is loaded from disk in under 0.1 seconds, eliminating the embedding cost entirely.

5.1.2 LLM Input Batching (Candidate Cap)

Passing all 10 retrieval candidates to the LLM would produce prompts exceeding 12,000 tokens, regularly truncating the JSON response. Candidates are capped at 7, keeping the total prompt under 9,000 tokens with 4,000 tokens reserved for the response. This implicit prompt caching strategy reduces both cost and truncation errors.

5.1.3 JSON Repair for Truncation Resilience

A `repair_json()` function handles cases where the LLM response is truncated mid-object. It recovers the last complete JSON object from the cut-off response, ensuring the pipeline never crashes on a partial response.

5.1.4 Rule Engine First

Running the deterministic rule engine before the LLM call is itself an optimisation: plans eliminated by hard constraints never consume LLM tokens. In the worst case (no hard-constraint violations), this saves zero tokens, in the best case (several plans eliminated), it reduces prompt length by hundreds of tokens per eliminated plan.

5.2 Performance: TTFT, TPOT, and Throughput

Performance was measured across five representative queries (see Table 5). Key observations:

- **TTFT of 0.33 s** reflects Groq's LPU architecture, which minimises model loading time.
- **TPOT of 0.0021 s/token** corresponds to 469 tokens/s, consistent with Groq's published throughput for Llama 3.3 70B.
- **Retrieval latency of 68.5 ms** is negligible relative to LLM latency (2.34 s), confirming that retrieval is not the bottleneck.
- **Total query time of 2.41 s** is well within acceptable interactive response times for a recommendation tool.

5.3 Observability: Logs and Traces

Structured logging is implemented throughout the pipeline. Each query produces a log trace covering: query receipt, profile extraction result, retrieval candidate list (with RRF scores), rule engine decisions (pass/fail per plan with reason), LLM call parameters, LLM response (raw JSON), repair actions taken (if any), and final ranked output.

Example log trace (abbreviated):

Listing 4: Example observability log for a single query

```
[INFO]  Query received: "I'm 68 with Parkinson's..."
[INFO]  Profile extracted: age=68, conditions=['neurological'],
   med_freq='daily'
[INFO]  Retrieval: 10 candidates in 0.071s
```

```
[INFO]  Rule engine: Plan D (Level Health) -> rule_score=1.00 [PASS]
[INFO]  Rule engine: Health Access (Vhi) -> rule_score=0.41 [PASS, low util score]
[INFO]  LLM call: 7 plans, ~8800 tokens, model=llama-3.3-70b-versatile
[INFO]  LLM latency: 2.34s, TTFT=0.33s, 944 output tokens
[INFO]  JSON repair: not required
[INFO]  Final ranking: Plan D (8/10), BeneFit (7/10), ...
[INFO]  Total query time: 2.41s
```

These logs are written to a rotating file (`query.log`) and are used during development to diagnose ranking anomalies, identify plan lookup failures, and monitor for LLM response truncation events.

6 Iteration Log & Conduct Summary

6.1 Practical Sessions Log

Table 6: Timeline of project progress across practical sessions.

Session	Progress	Problems Identified & Resolved
Week 1-2	Problem scoping, sourced Irish health insurance plan PDFs from all four providers	Data heterogeneity: providers use inconsistent terminology and document structures. Resolved by manual normalisation during chunking.
Week 3-4	Built initial BM25-only retrieval pipeline, constructed JSONL dataset	Keyword retrieval missed semantically relevant chunks (e.g., "specialist" vs. "consultant"). Added FAISS semantic layer.

Continued on next page

Table 6: Timeline of project progress across practical sessions (continued).

Session	**Progress**	**Problems Identified & Resolved**
Week 5-6	Built structured superset, implemented rule engine, integrated FAISS + BM25 with RRF	Plan key mismatches between superset and metadata. Resolved with `_build_plan_lookup()` and four-step fallback.
Week 7-8	Integrated LLM reranker, tuned prompt for JSON output, implemented JSON repair	LLM response truncation on 10-plan prompts. Resolved by capping at 7 plans.
Week 9-10	Built HTML/JS frontend, implemented performance metrics, evaluation runs	Frontend CORS issues during local dev. Resolved by serving frontend and backend from same origin.
Week 11	Evaluation with AI judge, documentation, report writing	Neurological condition recall gap identified in evaluation. Documented as known limitation, flagged for structured-flag improvement.

6.2 Team Contributions

Table 7: Individual team member contributions.

Member	Contributions
Ashwin Prasanth	Overall system architecture, FAISS + BM25 retrieval pipeline, RRF fusion, FAISS index management, code integration and repository maintenance.
Konstantinos Sklavenitis	Rule engine design and implementation, utilisation scoring formula, hard flag filter logic, structured superset construction and validation.
Kiran Meenakshi Sundaram	Data engineering, PDF chunking pipeline, metadata curation, JSONL dataset preparation, data cleaning and deduplication.
Charalampos Theodoridis	LLM integration, prompt engineering (reranker and profile extractor), JSON repair utility, evaluation framework, AI judge implementation, frontend development.

References

[1] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv preprint arXiv:2309.15217.

[2] Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333–389.

[3] Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-Scale Similarity Search with GPUs*. IEEE Transactions on Big Data, 7(3), 535–547.

[4] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. Proceedings of the 32nd ACM SIGIR Conference, 758–759.

[5] Meta AI. (2024). *Llama 3: Open Foundation and Fine-Tuned Chat Models*. https://llama.meta.com/llama3/

[6] Mixedbread AI. (2024). *mxbai-embed-large-v1: A High-Performance Text Embedding Model*. https://www.mixedbread.ai/blog/mxbai-embed-large-v1

[7] Health Insurance Authority Ireland. (2026). *Compare Health Insurance Plans*. https://www.hia.ie/consumer-information/compare-health-insurance-plans

[8] Groq Inc. (2025). *Groq LPU Inference Engine Documentation*. https://console.groq.com/docs

Appendix A: Repository Structure

GitHub Repository: https://github.com/AshwinPrasanth/ML-Deployment

Listing 5: Repository directory structure

```
ML-Deployment/
        Data-Engineering/          # Data curation notebooks and scripts
        Documentation/             # Architecture diagrams and notes
        Frontend/                  # HTML/JS frontend
        Policy/                    # Raw policy documents
        jsonl_datas/               # JSONL dataset files
              rag_chunks.jsonl
              MASTER_STRUCTURED_SUPERSET_2026-1.jsonl
              metadata (1).json
        smart_rag_with_rules.py    # Main inference engine
        Practical2.pdf             # Practical session 2 submission
        readme.md
        LICENSE                    # Apache 2.0
        NOTICE
```

Appendix B: Supported Providers and Plans

Table 8: Health insurance providers and plans covered in the dataset.

Provider	Plans
Irish Life Health	First Cover, BeneFit, Horizon 4, Health Plan 26.1, Select Starter, Health Plans Membership Handbook, Tailored Health Plans Membership Handbook
Laya Healthcare	Inspire, Prime Plan, First & Family Plan, PMI Plan B
Level Health	Plan A, Plan B (€150 Excess), Plan B (€300 Excess), Plan C, Plan D
Vhi	Company Plan Plus Level 1, Health Access, Hospital Plans Rules

Appendix C: Rule Engine Scoring Formulas

Utilisation intensity:

$$I = \frac{1}{N} \sum_{s \in S} f(s)$$

where S is the set of provided signals (medication frequency, specialist visits, hospital admissions) and $f(s) \in [0, 1]$ is the normalised signal value.

Utilisation score:

$$U = e_s \cdot (0.4I + 0.1) + d_s \cdot (0.3I + 0.05) + c_s \cdot 0.25 + v_s \cdot 0.20$$

where:

$$\begin{aligned}
e_s &= \frac{1}{1 + \bar{x}/250} \quad \text{(excess score, lower excess is better)} \\
d_s &= \min\big(\bar{d}/100,\ 1\big) \quad \text{(day-limit score)} \\
c_s &= \min(\bar{c}/100{,}000,\ 1) \quad \text{(benefit cap score)} \\
v_s &= \bar{p}/100 \quad \text{(coverage percentage score)}
\end{aligned}$$